

D4.1 Robot Sensing Migration

Due date: **01/03/2026**
Submission Date: **01/06/2026**
Revision Date: **14/08/2026**

Start date of project: **01/07/2026**

Duration: **12 months**

Responsible Person: **Yohannes Haile**

Revision: **1.4**

Project funded by Upanzi Network Dissemination Level		
PU	Public	PU
PP	Restricted to other programme participants	
RE	Restricted to a group specified by the consortium	
CO	Confidential, only for members of the consortium	

Executive Summary

Work Package 4 (WP4) addresses the migration of all software modules from [Work Package 4](#) of the CSSR4Africa project from ROS1 to ROS2. The objective is to ensure full compatibility with the ROS2 framework while making necessary changes needed to improve up on the original ROS1 implementations. This work package includes the complete migration of all modules developed in WP4, encompassing the following deliverables:

- Person Detection and Localization Module
- Face Detection and Mutual Gaze Localization Module
- Speech Event package (comprising the Speech Event node and the Speech Localization node)

Robot Localization will be developed directly for ROS2 and hence is not included in this migration deliverable. Sensor Tests isn't included in this deliverable to make the migration process more manageable considering the number of modules that need to be migrated also considering the number of individuals working on the entire project. Furthermore, unit tests for each nodes isn't included as well as the migration process is focused on the main functionality of the nodes.

Contents

1	Introduction	4
2	Requirements Definition	5
3	Migration Overview	7
4	Module Specifications	11
5	Module Design	12
6	Implementation	18
	References	29
	Principal Contributors	30
	Document History	32

1 Introduction

This document describes the migration of the [Work Package 4](#) software modules from ROS1 to ROS2 as part of the CSSR4Africa project. The CSSR4Africa project aims to develop culturally sensitive social robots capable of interacting naturally with people in African contexts, using the Pepper robot as the primary platform. The overall system architecture for the CSSR4Africa project is defined in [CSSR4Africa Deliverable D3.1](#) [1]. Work Package 4 covers the perception capabilities of the robot, including person detection and localization, face and mutual gaze detection, sound detection and localization, and speech event detection. These modules were originally developed for ROS1 (Noetic) and are being migrated to ROS2 (Humble) [2] to align with the project's long-term software strategy and to benefit from the improved reliability, security, and tooling offered by ROS2.

The migration encompasses the following deliverables:

- **D4.2.1:** Person Detection and Localization, which detects and localizes objects in the robot's field of view using RGB-D camera input. The module uses YOLOv11 [3] for detection and allows the user to specify which person class(es) to track (e.g., person, or any class supported by YOLO). Tracking is performed using BYTETrack [4].
- **D4.2.2:** Face and Mutual Gaze Detection and Localization, which detects faces, tracks them across frames, and determines whether mutual gaze is established between the robot and a user.
- **D4.3.2:** Speech Event. The Sound Detection and Localization module (D4.2.3) and the Speech Event module (D4.3.2) have been consolidated into a single ROS2 package. The package provides two nodes: the *Speech Recognition* node and the *Speech Localization* node. The Speech Recognition node replaces the original keyword-spotting approach with a full speech recognition and localization-in-time pipeline based on Voice Activity Detection (VAD) and Automatic Speech Recognition (ASR), detecting speech onset and offset and producing verbatim transcriptions. In addition, it comprises of the sound detection signal processing to reduce noise. The Speech Localization node estimates the horizontal direction of any sound source using SRP-PHAT beamforming on Pepper's four-microphone array.

Each module has been rewritten to conform to the ROS2 API, replacing ROS1 constructs with their ROS2 equivalents. This includes migrating from `rospy` to `rclpy`, from `catkin` to `ament_python` build system, from `.launch` XML files to Python-based ROS2 launch files, and from ROS1 `.msg` definitions to ROS2 interface packages. The migrated nodes maintain the same functional specifications as their ROS1 counterparts, ensuring continuity in system behavior while operating within the ROS2 ecosystem.

2 Requirements Definition

This section defines the migration requirements for Work Package 4 (WP4). The primary requirement is that each migrated ROS2 node must preserve the functional behavior of its ROS1 counterpart. In addition, the migration must satisfy the following technical and structural requirements.

ROS2 Compatibility

- All nodes must be implemented using `rclpy` and conform to the ROS2 (Humble) API.
- Build system files must be migrated from `catkin` to `ament_python`, with `package.xml` updated to format 3.
- Launch files must be rewritten as Python-based ROS2 launch files, replacing ROS1 XML `.launch` files.
- Custom message types must be redefined as ROS2 interface packages using `rosidl`.

Functional Preservation

- Each migrated node must subscribe to and publish on topics that are functionally equivalent to those in the original ROS1 implementation or has improved functionality. For example, the face detection node (D4.2.2) now relies on person detections from the person detection module (D4.2.1) for improved tracking accuracy, which is an intentional change to enhance performance.
- The detection algorithms and output data structures for each module must be preserved. Where tracking logic has been revised to improve accuracy (e.g., D4.2.2), those improvements are documented as intentional changes within this migration.

Module-Specific Requirements

- **D4.2.1 — Person Detection and Localization:** The migrated node must detect and localize objects in the robot's field of view using YOLOv11, with the target person class configurable by the user (e.g., person, or any class supported by YOLO). Tracking must be performed using BYTETrack, and the node must publish bounding box coordinates and depth estimates over the corresponding ROS2 topic.
- **D4.2.2 — Face and Mutual Gaze Detection and Localization:** The migrated node must detect faces, estimate head pose, and publish mutual gaze status and bounding box data over the corresponding ROS2 topic. SORT tracking has been removed; face tracking now relies on person detection from the person detection module (D4.2.1, person class) for improved tracking accuracy. A model warm up step must be included to pre-load detection models into memory before processing begins.
- **D4.3.2 — Speech Event Package (D4.2.3 + D4.3.2):** The Sound Detection and Localization module (D4.2.3) and the Speech Event module (D4.3.2) are delivered as two primary ROS2 nodes within the same package.

- The *Speech Recognition* node perform speech recognition that is it detects speech onset and offset using Voice Activity Detection, and then produce a verbatim transcription of the detected utterance using Automatic Speech Recognition. It must operate either as a ROS2 action server, returning the transcription as the action result when queried, or in a continuous standalone mode that publishes transcriptions to a topic. The original keyword-spotting approach had insufficient accuracy and has been replaced with a Silero VAD + Faster-Whisper pipeline.
- The *Speech Localization* node must detect sound events in the audio stream and publish the estimated horizontal direction (azimuth) of the sound source, along with a confidence score, using SRP-PHAT beamforming on Pepper's four-microphone planar array. In the ROS1 implementation, it was limited to frontal field of view of the robot in addition it utilized GCC-PHAT for localizing the speech.

Configurable Parameters

- Each node must load its operating parameters from its respective YAML configuration file. Parameters are declared as ROS2 node parameters and loaded via a `--params-file` argument at launch time.
- Verbose mode must remain supported across all modules, providing optional diagnostic output and visual debugging.

Misalignment and Scope Exclusions

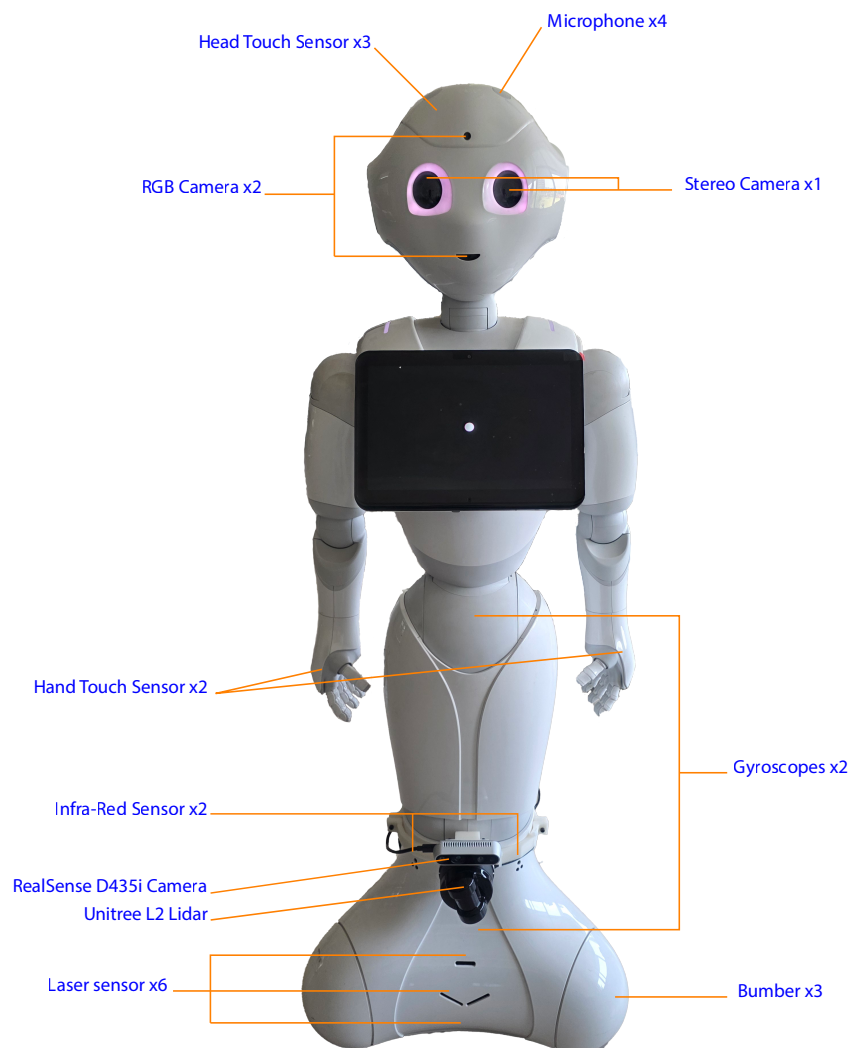
- Unit tests for each module are not included in this deliverable; the migration is focused on the functional nodes.
- The D4.1 Sensor Tests and D4.2.4 Robot Localization modules are excluded from this migration. Robot Localization will be developed natively for ROS2 using Self Localization and Mapping (SLAM).
- The simulator is not supported by the migrated modules; all nodes target the physical Pepper robot with the Intel RealSense D435i camera.

3 Migration Overview

This section summarises the systematic changes applied across all modules during the ROS1-to-ROS2 migration.

Robot Platform and Sensors

Figure 1 shows the sensors available on the robot platform, including Pepper's onboard RGB and stereo cameras, microphones, head and hand tactile sensors, bumpers, gyroscopes, and infrared/laser range sensors. It also indicates the mounting location, at the base of the robot, of two sensors added for this work package: an external Intel RealSense D435i RGB-D camera (see Section 4) and a Unitree L2 LiDAR. Detailed diagrams of these two mounted sensors will be added as further subfigures of Figure 1 in a future revision of this document.



(a) Sensor layout on the Pepper robot, including the mounted RealSense D435i and Unitree L2 LiDAR.

Figure 1: Sensors available on the robot platform.

LiDAR and RealSense Sensor Mount

This subsection documents the custom mount used to add the Unitree L2 LiDAR and Intel RealSense D435i camera to Pepper.

Purpose

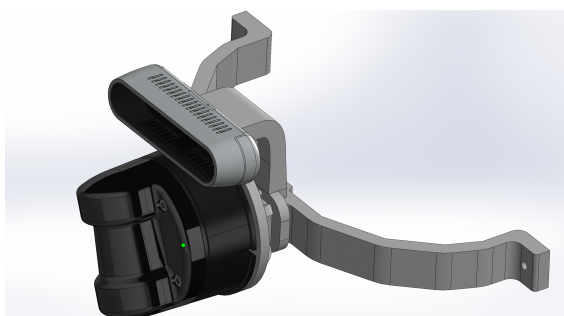
The mount provides a single, rigid attachment point for the Unitree L2 LiDAR and Intel RealSense D435i camera on Pepper's base, extending the robot's sensing capability beyond its built-in sensors.

Mounting Interface

The mount attaches around the circumference of Pepper's base/hip section as a contoured band, secured with a clasp fastener. It requires no drilling or other permanent modification to the robot's shell. The Intel RealSense D435i and Unitree L2 LiDAR are each secured to the mount bracket using 8 mm bolts and matching nuts.

Isometric Render and Physical Assembly

Figure 2 shows the CAD isometric render of the mount with the Unitree L2 LiDAR and Intel RealSense D435i in place, alongside a photograph of the assembled mount installed on the physical Pepper robot.



(a) Isometric CAD render of the mount assembly.



(b) Physical mount installed on the Pepper robot.

Figure 2: LiDAR and RealSense mount: CAD render and physical assembly.

Material

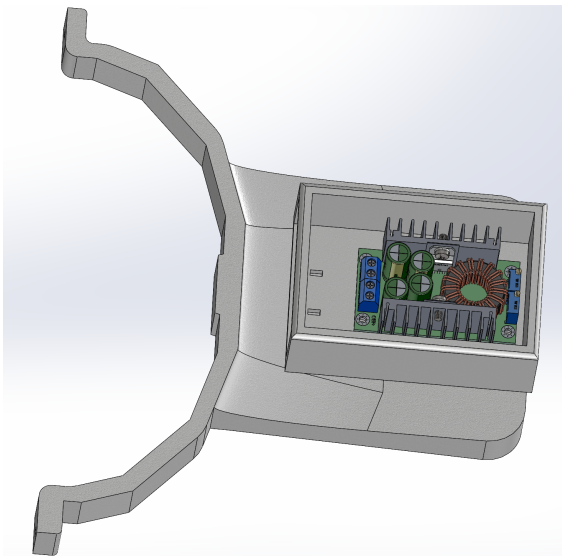
The mount is 3D printed in ABS.

Power Supply

The Unitree L2 LiDAR is powered from an external power bank via a 300 W XL4016 DC-DC buck converter, which steps down the power bank's output voltage to the level required by the LiDAR and feeds it directly. Figure 3(a) shows the converter board housed in an enclosure integrated into the mount assembly.

Companion Electronics

Figure 3(b) shows the power bank that supplies the buck converter described above, together with an NVIDIA Jetson Nano companion computer, mounted on the back of the robot's torso.



(a) 300 W XL4016 DC-DC buck converter housed in the mount enclosure.



(b) Power bank and NVIDIA Jetson Nano mounted on the back of the robot.

Figure 3: Buck converter and companion electronics powering the LiDAR.

Common Migration Changes

Module	Migration Approach	Redesigned?
D4.2.1 Person Detection	Direct port with API updates; tracker upgraded to BYTETrack; Migrated to use YOLOv11 replacing YOLOv8	Partial
D4.2.2 Face & Mutual Gaze	Direct port; SORT removed, tracking delegated to D4.2.1	Partial
D4.2.3 & D4.3.2 – Speech Event (combines Sound Detection and Speech Event from ROS1)	The standalone Sound Detection module (D4.2.3) is removed; its functionality is absorbed into the unified <code>speech_event</code> package. The Speech Recognition node was fully redesigned with a VAD + ASR pipeline (Silero VAD + Faster-Whisper) and a ROS2 Action-based interface. The Speech Localization node was ported using SRP-PHAT beamforming.	Yes

Table 1: Per-module migration summary.

4 Module Specifications

This section describes the expected behaviour and functional requirements of each module delivered in Work Package 4.

D4.2.1 — Person Detection and Localization

The Person Detection and Localization node (originally specified in [CSSR4Africa Deliverable D4.2.1](#) [5]) must detect and localise people in the robot's field of view in real time using an RGB-D camera. It is expected to accept a configurable target class list, so that the operator can restrict detection to `person` or any other COCO class. Detections must be associated with persistent track identities across consecutive frames so that downstream modules can follow individuals over time. For each tracked object, the node must estimate a 3D position (pixel centroid plus depth) and publish it for consumption by other modules such as face detection. The node must support both the Intel RealSense D435i and Pepper's built-in cameras, selectable via a configuration parameter, and must operate in verbose mode for diagnostic purposes when required.

D4.2.2 — Face and Mutual Gaze Detection and Localization

The Face and Mutual Gaze Detection node (originally specified in [CSSR4Africa Deliverable D4.2.2](#) [6]) must detect faces in the robot's camera feed, estimate head pose for each detected face, and determine whether mutual gaze is established between the robot and the user. It is expected to consume person detections from D4.2.1 to associate faces with tracked individuals, removing the need for an independent tracking module. The node must publish, for each detected face, the bounding box, estimated head pose angles, and a mutual-gaze flag. Mutual gaze must be determined by comparing the absolute values of yaw, pitch, and roll against a configurable angular threshold. The node must support verbose mode for debugging.

D4.3.2 — Speech Event Package

The Speech Event package (originally developed across [CSSR4Africa Deliverable D4.2.3](#) and [CSSR4Africa Deliverable D4.3.2](#)) must deliver two complementary audio-processing capabilities within a single ROS2 package. The standalone Sound Detection module (D4.2.3) is removed; sound detection signal processing is instead absorbed into this package.

The *Speech Recognition* node must replace the original keyword-spotting approach with a full speech recognition pipeline that detects speech onset and offset (using Voice Activity Detection) and produces verbatim transcriptions (using Automatic Speech Recognition). It must support two operating modes: an action server mode, in which transcription is triggered on demand by another module, and a continuous standalone mode, in which the node monitors audio indefinitely and publishes transcriptions as they occur. Audio processing must be enable/disable-able at runtime via a service interface.

The *Speech Localization* node must estimate the horizontal azimuth of any sound source from Pepper's four-microphone array and publish it continuously. Estimates must include a confidence score, and results falling below a configurable confidence threshold must be discarded. The node must run in parallel with the Speech Recognition node, both consuming the same raw audio stream from Pepper's microphone array.

5 Module Design

This section describes how each module is implemented at the code level, covering node architecture, data flow, and key algorithmic components.

D4.2.1 — Person Detection and Localization

Node Architecture

The package follows a C++ two-file split that is used consistently across all compiled modules in this work package. `person_detection_application.cpp` is the ROS2 entry point: via the shared `dec_common::runNode<>()` helper, it initialises `rclcpp`, constructs the `Yolov11Node` lifecycle node class defined in `person_detection_implementation.cpp`, spins it, and shuts it down cleanly on `SIGINT`. `Yolov11Node` derives from `PersonDetectionNode`, which in turn derives from the shared `dec_common::CameraLifecycleNode` base (also used by face detection) that owns camera topic resolution, synchronised subscriptions, depth decoding, and debug visualisation. All detection, tracking, and publishing logic specific to person detection lives in `person_detection_implementation.cpp`.

As a ROS2 lifecycle node, initialisation is split across two transitions rather than done all at once in the constructor. On `configure`, the node declares and reads its ROS2 parameters, creates its lifecycle publishers, and resolves the target class list; `Yolov11Node` additionally loads the YOLOv11 ONNX model into an ONNX Runtime C++ [?] inference session (attempting a CUDA execution provider first, falling back to CPU) and instantiates the `dec_common::byte_tracker::ByteTrack` tracker, a native C++ port of the ByteTrack algorithm (no longer the Python supervision [7] library). A warm-up pass over a zero-valued dummy input is run immediately after model loading to pre-load weights into memory and avoid first-frame latency. On `activate`, the synchronised RGB and depth subscribers are created (via `ApproximateTimeSynchronizer`) and the image-timeout monitor is started; `deactivate` and `cleanup` tear these down and release the ONNX session in reverse order.

Per-Frame Data Flow

Each time a synchronised pair of colour and depth images arrives, the callback executes the following pipeline:

1. The incoming `sensor_msgs/Image` (or `CompressedImage`) is decoded into an OpenCV BGR array using `cv_bridge`.
2. The BGR array is converted to RGB, resized to the model's required input resolution, normalised to $[0, 1]$, transposed from *HWC* to *CHW* layout, and wrapped in a batch dimension to produce a float32 tensor of shape $[1, 3, H, W]$.
3. The tensor is passed to the ONNX Runtime session. The raw $[1, 84, 8400]$ output (4 box coordinates plus 80 per-class logits, sigmoid-activated, for 8400 candidate predictions) is post-processed to extract bounding boxes in $[x_1, y_1, x_2, y_2]$ format, class probabilities, and confidence scores, followed by per-class non-maximum suppression. Detections below `confidence_threshold` and those not in `target_classes` are discarded.

4. The surviving detections are wrapped in a `byte_tracker::Detections` struct and passed to `tracker_->updateWithDetections()`. As shown in Figure 4, ByteTrack first matches high-confidence detections to existing tracks by IoU (Hungarian algorithm), then re-evaluates lower-confidence detections against still-unmatched tracks. Detections that receive a `tracker_id` are kept; transient detections without a stable track are dropped.
5. For each tracked bounding box, the pixel centroid ($\bar{u}, \bar{v}$) is computed as the midpoint of the box. A region of interest (ROI) of 10% of the box width and height centred on the centroid is extracted from the depth image. All valid (non-zero, finite) depth readings inside the ROI are averaged (person detection uses the mean, not the median) and converted from millimetres to metres to produce the z -coordinate of the 3D centroid.
6. The results are packed into a `dec_interfaces/PersonDetection` message and published on `/person_detection/data`. If `verbose_mode` is enabled, annotated debug images are published to `/person_detection/debug` and `/person_detection/depth_debug`.

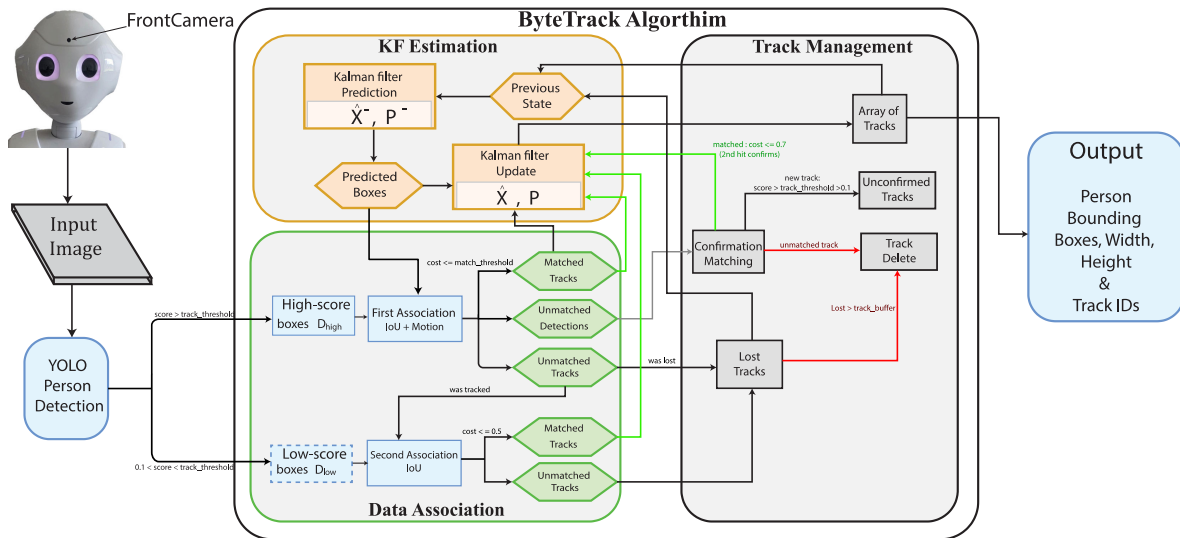


Figure 4: Overview of the ByteTrack multi-object tracking algorithm used to associate detections with persistent track identities.

Camera Abstraction

The node supports three camera sources selected at configuration time, resolved by the shared `dec_common::CameraLifecycleNode` base against `pepper_topics.yaml`. When camera is set to `realsense`, the node subscribes to the RealSense colour and pixel-aligned depth topics; the RealSense driver is launched separately (see Launch File below) with `align_depth.enable: True` so that every depth pixel corresponds exactly to its colour-image counterpart. When camera is set to `pepper`, the node subscribes to the NAOqi driver's front and depth camera topics. Any other value leaves no camera driver started, allowing the node to consume data replayed from a ROS2 bag file.

D4.2.2 — Face and Mutual Gaze Detection and Localization

Node Architecture

The package follows the same C++ two-file pattern as D4.2.1. `face_detection_application.cpp` is the entry point, running the `SixDrepNet` lifecycle node via `dec_common::runNode<>()` and invoking its `cleanup()` method (closing any OpenCV debug windows) on shutdown. `face_detection_implementation.cpp` contains the node classes: `SixDrepNet` derives from `FaceDetectionNode`, which (like `PersonDetectionNode`) derives from the shared `dec_common::CameraLifecycleNode` base. `SixDrepNet` hosts two ONNX-based models, a YOLOONNX wrapper around the GoldYOLO [9] face detector and a `SixDrepNet` [8] head-pose estimator, and subscribes to both the camera stream and the person detections published by D4.2.1 (when `require_person_detection` is enabled). When it is disabled, `FaceDetectionNode` instead runs its own dedicated `byte_tracker::ByteTrack` instance to track faces directly, so the module can still operate without D4.2.1 running.

On configure, the node declares/reads its ROS2 parameters, creates its lifecycle publishers, and (for `SixDrepNet`) instantiates the two ONNX Runtime sessions (`face_detection_goldYOLO.onnx` and `face_detection_sixdrepnet360.onnx`), each with a warm-up pass and a CUDA-then-CPU execution provider fallback. On activate, it subscribes to `/person_detection/data` (if person detection is required) and creates the synchronised RGB-D camera subscriptions; deactivate and cleanup tear these down and release the ONNX sessions in reverse order.

Per-Frame Data Flow

On each synchronised image pair, one of two code paths executes depending on `require_person_detection`:

1. The colour image is decoded and passed to the GoldYOLO face detector (`YOLOONNX::detect`). NMS is baked into the exported GoldYOLO graph, so post-processing here is only a confidence filter and coordinate rescale to image-pixel space; detections narrower or shorter than 20 px, or falling outside the image bounds, are also discarded.
2. **When `require_person_detection` is true** (`processFrameWithPersonDetection`): the latest `/person_detection/data` snapshot is checked for staleness against `person_detection_timeout` and discarded if too old. Surviving faces are matched to `person` class detections using the Hungarian algorithm over a hand-tuned cost function combining normalised face-to-person-centroid distance, a penalty for faces positioned low in the person's bounding box, a penalty if the face-to-person height ratio falls outside `[0.1, 0.4]`, and detector confidence, with a greedy nearest-person fallback for any face the Hungarian assignment leaves unmatched (e.g., two faces in one person box). Face identity is inherited from the person track ID assigned by `ByteTrack` in D4.2.1, so no independent face tracker is needed on this path.
When `require_person_detection` is false (`processFrameStandalone`): faces are tracked directly by the node's own `byte_tracker::ByteTrack` instance, and face identity is the tracker's own ID (`face_<id>`).
3. Each matched/tracked face bounding box is cropped from the colour image, resized to `224 × 224` pixels, and passed to `SixDrepNet`, which outputs a 6D rotation representation. The node

converts this representation to a 3×3 rotation matrix and then decomposes it into Euler angles: yaw (left–right rotation), pitch (up–down tilt), and roll (in-plane rotation). A face is flagged as in mutual gaze when `|yaw|` and `|pitch|` both lie within the `sixdrepnet_headpose_angle` threshold; roll is computed but is not part of the mutual-gaze test.

4. Depth is sampled from the aligned depth image using the same ROI-averaging approach as D4.2.1 (face detection uses the median, not the mean, within the ROI). On the person-detection path, `prioritize_face_depth` selects whether the face-region depth or the person-region depth (from D4.2.1’s centroid) is preferred when both are available, with the other used as a fallback.
5. Results are packed into a `dec_interfaces/FaceDetection` message, containing the face ID, 3D centroid (image coordinates plus depth), bounding box width and height, and mutual-gaze flag, and published on `/face_detection/data`.

D4.3.2 — Speech Event Package

Package Overview

The `speech_event` package hosts three ROS2 nodes that share the same audio source: `speech_recognition`, `sound_localization`, and `audio_recorder`. All three subscribe to `/naoqi_driver/audio` (`naoqi_bridge_msgs/AudioBuffer`), which carries interleaved four-channel audio from Pepper’s microphone array at 48 kHz. Each `AudioBuffer` message contains 4096 samples per channel (≈ 85 ms of audio). Of the three, only `speech_recognition` is a ROS2 lifecycle node (`rclpy.lifecycle.LifecycleNode`); `sound_localization` and `audio_recorder` are plain nodes. All three nodes are launched together by `speech_event.launch.py`, operate in separate threads, and do not share state.

Speech Recognition Node

The `speech_recognition` node is implemented in `speech_event_implementation.py`. Its design centers on a VAD state machine that gates a transcription pipeline.

Audio ingestion. On each `/naoqi_driver/audio` callback, the interleaved four-channel PCM buffer is de-interleaved and the first channel is extracted as a mono signal. The 48 kHz signal is downsampled to 16 kHz using a polyphase resampler (`scipy.signal.resample_poly`) [?] before any further processing.

Voice Activity Detection. The 16 kHz audio is segmented into 512-sample chunks and fed to an `OnnxWrapper` that wraps the Silero VAD [10] ONNX model. Silero VAD is a compact recurrent neural network that outputs, for each chunk, the probability that speech is present. A two-threshold hysteresis state machine governs the speech/silence boundary: a speech segment begins when the probability exceeds `speech_threshold` (default 0.7), and ends only after the probability remains below `neg_threshold` (default 0.35) for at least `min_silence_duration_ms` (default 800 ms). A pre-speech buffer of `pre_speech_buffer_ms` (default 200 ms) is prepended to each detected segment to avoid clipping the onset. Segments shorter than `min_speech_duration` (0.3 s) are discarded; segments longer than `max_speech_duration_s` (10 s) are truncated.

Transcription. Once a complete speech segment is confirmed, the 16 kHz buffer is submitted to a `ThreadPoolExecutor` to avoid blocking the ROS2 executor. The transcription is performed by Faster-Whisper [11] (a CTranslate2 reimplementation of OpenAI Whisper [12]), which runs

with float16 compute type on CUDA when available. The model ID (deepdml/faster-whisper-large-v3-turbo-ct2 by default) is configurable via `whisper_model_id`. When `noise_cleaning_enabled` is true, the confirmed speech buffer is first passed through the `SpeechDenoiser` class (implemented in `speech_event_denoiser.py`), which applies bandpass filtering, harmonic notch filtering, and Wiener-filter spectral subtraction using a pre-recorded noise profile (`data/noise_profile.npy`) before the buffer is submitted for transcription.

Operating modes. The `action_server` parameter selects between two modes. In action server mode, the node exposes a `dec_interfaces/action/SpeechRecognition` action server on `/speech_recognition`; audio processing is active only while a goal is held, and the transcription is returned as the action result. In standalone/continuous mode, the node processes audio indefinitely and publishes each transcription to `/speech_event/text`. A `std_srvs/SetBool` service on `/speech_event/set_enabled` allows audio processing to be paused and resumed at runtime in either mode. When `vad_always_active` is true, the VAD state machine continues running even while no action goal is active, so that speech onset is not missed at the start of a goal.

Speech Localization Node

The `sound_localization` node is implemented in `speech_event_localization.py`. It estimates the horizontal azimuth of a sound source using Steered Response Power with Phase Transform (SRP-PHAT) beamforming via the `pyroomacoustics` [13] library.

Microphone geometry. Pepper's four microphones are modelled as a planar array with known XYZ coordinates. This geometry is passed to `pyroomacoustics` to compute the theoretical inter-microphone time delays of arrival for each candidate direction.

SRP-PHAT computation. On each audio callback, the 48 kHz four-channel buffer is passed through a Hann-windowed STFT with FFT size 1024. For each candidate azimuth θ in a grid of 36 uniformly spaced angles, the algorithm evaluates

$$P(\theta) = \sum_{m \neq n} \int_{f_{\min}}^{f_{\max}} \frac{G_{mn}(f)}{|G_{mn}(f)|} e^{j2\pi f \tau_{mn}(\theta)} df$$

where $G_{mn}(f)$ is the cross-power spectral density between microphones m and n and $\tau_{mn}(\theta)$ is the theoretical delay for direction θ at the speed of sound $c = 343$ m/s. The integration is restricted to a configurable band (default 500–2500 Hz) to avoid low-frequency noise and spatial aliasing. The azimuth that maximizes $P(\theta)$ is selected as the estimated source direction.

Confidence and filtering. A confidence score $(\text{peak} - \mu)/\mu$ (where μ is the spatial spectrum mean) is computed for each estimate. Estimates below `confidence_threshold` are discarded. An RMS amplitude gate (`intensity_threshold`) pre-filters silent frames before the SRP-PHAT computation. Optional circular-mean smoothing over a configurable history window reduces frame-to-frame jitter.

Published outputs. The node publishes the estimated azimuth (`std_msgs/Float32`), a unit direction vector (`geometry_msgs/Vector3Stamped`), the confidence score, an estimated source pose (`geometry_msgs/PoseStamped`), and an RViz arrow marker (`visualization_msgs/Marker`) for visualization.

Audio Recorder Node

The `audio_recorder` node is implemented in `speech_event_recorder.py`. It is a utility node intended for development and data collection: it subscribes to `/naoqi_driver/audio` and writes the incoming `naoqi\_bridge\_msgs/AudioBuffer` stream to disk as a multi-channel WAV file. When `split_channels` is `true`, one mono WAV file is written per microphone channel. The output path prefix, maximum recording duration, and channel-splitting behaviour are all configurable via the `audio_recorder` parameter group in `speech_event_configuration.yaml`. The node is not required for normal robot operation, but is launched alongside the other two nodes by `speech_event.launch.py` for convenience.

6 Implementation

D4.2.1 — Person Detection and Localization

File Organization

Figure 5 shows the file structure of the person detection package.

```

person_detection
├── config
│   └── person_detection_configuration.yaml
├── data
│   └── pepper_topics.yaml
├── include/person_detection
│   └── person_detection_interface.h
├── launch
│   └── person_detection.launch.py
├── models
│   └── person_detection_yolov11m.onnx
├── src
│   ├── person_detection_application.cpp
│   └── person_detection_implementation.cpp
├── test
│   └── test_bag_replay_launch.py
├── CMakeLists.txt
├── package.xml
└── README.md

```

Figure 5: File structure of the person detection package.

Configuration File

The operation of the person detection node is controlled by the YAML configuration file `person_detection_configuration.yaml`. The available key-value pairs are listed in Table 2.

Key	Value	Description
camera	realsense or pepper	Selects the camera source. Determines which ROS topics to subscribe to.
use_compressed	true or false	Whether to subscribe to compressed image topics. Compressed depth (compressedDepth) is used for RealSense; not supported for Pepper.
image_timeout	<number>	Time (seconds) after which a warning is logged if no images have been received.
verbose_mode	true or false	Enables diagnostic logging, OpenCV debug windows (if a display is available), and publication of debug image topics.
confidence_threshold	<number>	Minimum YOLOv11 detection confidence (0.0–1.0) required to pass a detection to the tracker; also reused as the per-class NMS IoU threshold.
target_classes	[<string>, ...]	List of COCO class names (or numeric indices) to track (e.g., [person]). Empty or [all] tracks all 80 classes.
track_threshold	<number>	ByteTrack activation threshold: minimum confidence for a detection to initialise or update an active track.
track_buffer	<integer>	Number of consecutive frames a track may remain unmatched before being removed.
match_threshold	<number>	IoU threshold used by ByteTrack when matching detections to existing tracks.
frame_rate	<integer>	Expected video frame rate (fps), used by ByteTrack to scale track buffer timing.

Table 2: Configuration file key-value pairs for the person detection node.

Launch File

The launch file `person_detection.launch.py` starts only the `person_detection` node itself; it no longer starts a camera driver. The camera feed is expected to be supplied separately, either by another launch file that brings up a shared camera (e.g., the overt attention system’s launch file, when person detection is run as a dependency of face detection or attention) or by replaying a ROS2 bag file; the node’s actual topic names, resolved from `pepper_topics.yaml` at runtime, are listed in Table 3 below.

Topics Subscribed

The person detection node subscribes to the following topics:

Camera	Topic Name	Message Type
RealSenseCameraRGB	/camera/color/image_raw_custom	sensor_msgs/Image
RealSenseCameraRGB (Compressed)	/camera/color/image_raw_custom/compressed	sensor_msgs/CompressedImage
RealSenseCameraDepth	/camera/aligned_depth_to_color/image_raw_custom	sensor_msgs/Image
RealSenseCameraDepth (Compressed)	/camera/aligned_depth_to_color/image_raw_custom/compressedDepth	sensor_msgs/CompressedImage
PepperFrontCamera	/pepper/front/image_raw	sensor_msgs/Image
PepperDepthCamera	/camera/depth/image_raw	sensor_msgs/Image

Table 3: Topics subscribed by the person detection node.

The colour and depth topics are subscribed using an `ApproximateTimeSynchronizer` with a synchronisation slop of 0.1 s (RealSense) or 5.0 s (Pepper) to account for timing differences between the two streams.

Topics Published

The person detection node publishes the topics listed in Table 4.

Topic Name	Message Type	Description
/person_detection/data	dec_interfaces/PersonDetection	Array of tracked objects with track IDs, class names, confidence scores, 2D+depth centroids, and bounding box dimensions.
/person_detection/debug	sensor_msgs/Image	Annotated RGB debug image (published when <code>verboseMode</code> is true).
/person_detection/depth_debug	sensor_msgs/Image	Colourised depth debug image (published when <code>verboseMode</code> is true).

Table 4: Topics published by the person detection node.

D4.2.2 — Face and Mutual Gaze Detection and Localization

File Organization

The source code for conducting face detection, mutual gaze detection, and localization is structured into two primary components: `face_detection_application` and `face_detection_implementation`. The `face_detection_implementation` component encapsulates all the essential functionality required for executing face detection and mutual gaze detection using SixDRepNet for head pose estimation. When person detection input is enabled (the default), face tracking relies on person detections received from the person detection node (D4.2.1) via Hungarian assignment, removing the need for a dedicated Centroid Tracker or SORT; when disabled, the node falls back to its own internal ByteTrack instance so it can still run independently. The `face_detection_application` component serves as the entry point, invoking the main functions to run the face detection node and executing the functions defined within `face_detection_implementation`.

Figure 6 shows the file structure of the face detection package. It also hosts a related but separate `age_gender_detection` node (age and gender estimation from the same face crops), which is outside the scope of this migration deliverable and not otherwise documented here.

```
face_detection
├── config
│   ├── age_gender_detection_configuration.yaml
│   └── face_detection_configuration.yaml
├── data
│   └── pepper_topics.yaml
├── include/face_detection
│   ├── age_gender_detection_interface.h
│   └── face_detection_interface.h
├── launch
│   ├── age_gender_detection.launch.py
│   └── face_detection.launch.py
├── models
│   ├── face_detection_goldYOLO.onnx
│   └── face_detection_sixdrepnet360.onnx
├── src
│   ├── age_gender_detection_application.cpp
│   ├── age_gender_detection_implementation.cpp
│   ├── face_detection_application.cpp
│   └── face_detection_implementation.cpp
├── CMakeLists.txt
├── package.xml
└── README.md
```

Figure 6: File structure of the face detection package.

Configuration File

The operation of the face detection node is determined by the contents of the configuration file, which contains the key-value pairs listed in Table 5. The configuration file is named `face_detection_configuration.yaml`.

Key	Value	Description
<code>camera</code>	<code>realsense, pepper</code>	Selects the camera source. Controls which image topics are subscribed to.
<code>use_compressed</code>	<code>true or false</code>	Whether to subscribe to compressed image topics.
<code>verbose_mode</code>	<code>true or false</code>	Enables diagnostic logging and OpenCV debug display windows.
<code>image_timeout</code>	<code><number></code>	Timeout (seconds) after which a warning is logged if no images are received.
<code>sixdrepnet_confidence</code>	<code><number></code>	Detection confidence threshold (0.0–1.0) for the GoldYOLO face detector used by SixDRepNet.
<code>sixdrepnet_headpose_angle</code>	<code><number></code>	Maximum angular deviation (degrees) in yaw and pitch for a face to be classified as in mutual gaze. Roll is computed but not tested.
<code>require_person_detection</code>	<code>true or false</code>	When <code>true</code> , the node subscribes to <code>/person_detection/data</code> and matches faces to person tracks from D4.2.1 via Hungarian assignment. When <code>false</code> , the node tracks faces directly with its own ByteTrack instance, running independently of D4.2.1.
<code>person_detection_timeout</code>	<code><number></code>	Maximum age (seconds) of person detection data before it is considered stale and discarded.
<code>prioritize_face_depth</code>	<code>true or false</code>	When <code>true</code> (default), prefers the face-region depth over the person-region depth when both are available; when <code>false</code> , the preference is reversed. Only applies when <code>require_person_detection</code> is <code>true</code> .

Table 5: Configuration file key-value pairs for the face detection node.

Input File

There is no input file for the face detection node.

Output File

There is no output file for the face detection node. The node uses OpenCV to display the detected faces with bounding boxes and labels, and the mutual gaze detection.

Launch File

The launch file `face_detection.launch.py` reads the `camera` value from `face_detection_configuration.yaml` at launch time and optionally starts the appropriate camera driver alongside the `face_detection` node; it no longer launches `person_detection`, which must be started separately (e.g., by the overt attention system's launch file, or manually) when `require_person_detection` is enabled. It exposes one launch argument:

- `launch_camera`: controls whether the camera driver is started by this launch file. Defaults to `false` (assume the camera topics are already available, e.g. from a shared camera launch or a ROS2 bag); set to `true` to have this launch file start the camera driver itself.

When `launch_camera:=true` and `camera` is `realsense` in the configuration file, the `realsense2_camera_node` is started with a 640×480 colour and depth profile at 15 fps, with depth alignment and time synchronisation enabled, and `BEST_EFFORT` QoS on image topics. When `camera` is `pepper`, the `naoqi_driver_node` is launched instead with the robot connection parameters embedded in the launch file.

Models

The face detection node uses two models for face detection and head pose estimation, listed in Table 6. The models are stored in the `models` directory.

Models	Description
<code>face_detection_goldYOLO.onnx</code>	YOLO-based face detection model.
<code>face_detection_sixdrepnet360.onnx</code>	SixDRepNet head pose estimation model.

Table 6: Models used by the face detection node.

Topics File

For the test, a selected list of the topics for the robot is stored in the topics file. The topic files are written in the `.yaml` file format. The data file is written in key-value pairs where the key is the camera and the value is the topic. The topics file for the robot is named `pepper_topics.yaml`.

Topics Subscribed

The face detection node subscribes to the topics listed in Table 7.

Camera	Topic Name	Message Type
RealSenseCameraRGB	/camera/color/image_raw_custom	sensor_msgs/Image
RealSenseCameraRGB (Compressed)	/camera/color/image_raw_custom/compressed	sensor_msgs/CompressedImage
RealSenseCameraDepth	/camera/aligned_depth_to_color/image_raw_custom	sensor_msgs/Image
RealSenseCameraDepth (Compressed)	/camera/aligned_depth_to_color/image_raw_custom/compressedDepth	sensor_msgs/CompressedImage
PepperFrontCamera	/pepper/front/image_raw	sensor_msgs/Image
PepperDepthCamera	/naoqi_driver/camera/depth/image_raw	sensor_msgs/Image

Table 7: Topics subscribed by the face detection node.

Topics Published

The face detection node publishes the topics listed in Table 8.

Topic Name	Message Type	Description
/face_detection/data	dec_interfaces/FaceDetection	An array of face records containing face labels, 3D image coordinates of the bounding box, width and height of the bounding box, and a boolean value indicating mutual gaze detection.
/face_detection/debug	sensor_msgs/Image	Annotated RGB debug image (published when verboseMode is true).
/face_detection/depth_debug	sensor_msgs/Image	Colourised depth debug image (published when verboseMode is true).

Table 8: Topics published by the face detection node.

D4.3.2 — Speech Event Package

File Organization

Figure 7 shows the file structure of the speech event package:

Configuration File

All three nodes share a single configuration file `speech_event_configuration.yaml`. Parameters are grouped under the ROS2 node name of each executable. The key-value pairs for the *Speech Recognition* node (`speech_recognition`) are listed in Table 9, and those for the *Speech Localization* node (`sound_localization`) are listed in Table 10.

```

speech_event
├── config
│   └── speech_event_configuration.yaml
├── data
│   ├── noise_profile.npy
│   └── pepper_topics.yaml
├── resource
│   └── speech_event
├── speech_event
│   ├── __init__.py
│   ├── speech_event_application.py
│   ├── speech_event_denoiser.py
│   ├── speech_event_implementation.py
│   ├── speech_event_localization.py
│   └── speech_event_recorder.py
├── package.xml
├── requirements.txt
├── setup.cfg
├── setup.py
└── README.md

```

Figure 7: File structure of the speech event package.

Key	Value	Description
microphone_topic	/naoqi_driver/audio	ROS2 topic name for the raw audio input stream.
input_sample_rate	48000	Sample rate (Hz) of the incoming audio from Pepper's microphone array.
sample_rate	16000	Target sample rate (Hz) after downsampling, required by Silero VAD and Whisper.
device	cuda or cpu	Inference device for Faster-Whisper. Use cuda for GPU acceleration.
compute_type	float16	Numerical precision for Faster-Whisper inference (float16 or int8).
language	en	Language code passed to Whisper for transcription.
whisper_model_id	<string>	Faster-Whisper model identifier (e.g., deepdml/faster-whisper-large-v3-turbo-ct2).
speech_threshold	0.7	VAD probability threshold above which speech onset is declared.
neg_threshold	0.35	VAD probability threshold below which silence accumulates toward segment end.
min_silence_duration_ms	800	Minimum silence duration (ms) required to close a speech segment.

continued on next page

Table 9 – continued from previous page

Key	Value	Description
min_speech_duration	0.3	Minimum speech segment duration (s); shorter segments are discarded.
max_speech_duration_s	10.0	Maximum speech segment duration (s); longer segments are truncated.
pre_speech_buffer_ms	200	Audio lookback (ms) prepended to each segment to avoid onset clipping.
intensity_threshold	0.001	RMS amplitude gate; frames below this value are discarded before VAD.
transcription_timeout_s	5.0	Maximum time (s) to wait for Whisper to finish transcribing a segment.
action_server	true or false	When true, the node runs as a ROS2 action server; when false, it publishes transcriptions continuously.
vad_always_active	true or false	When true, VAD runs even while no action goal is active.
noise_cleaning_enabled	true or false	Enables the SpeechDenoiser pipeline before Whisper transcription.
noise_profile_path	<path>	Path to a .npy file containing a pre-recorded noise magnitude spectrum.
noise_alpha	0.5	Blending weight (0.0–1.0) between the rolling noise estimate and the static profile.

Table 9: Configuration parameters for the speech recognition node.

Key	Value	Description
microphone_topic	/naoqi_driver/audio	ROS2 topic name for the raw audio input stream.
sample_rate	48000	Sample rate (Hz) of the incoming audio.
speed_of_sound	343.0	Speed of sound (m/s) used to compute inter-microphone time delays.
nfft	1024	FFT size for the Hann-windowed STFT used in SRP-PHAT.
angular_resolution	36	Number of candidate azimuth angles uniformly distributed over 360°.
freq_range_min	500	Lower bound (Hz) of the frequency band used in SRP-PHAT integration.
freq_range_max	2500	Upper bound (Hz) of the frequency band used in SRP-PHAT integration.
num_chunks_for_localization	6	Number of audio chunks accumulated per localization estimate.
update_rate_hz	2.0	Target rate (Hz) at which localization estimates are published.

continued on next page

Table 10 – *continued from previous page*

Key	Value	Description
confidence.threshold	0.15	Minimum confidence score for an azimuth estimate to be published.
intensity.threshold	0.001	RMS amplitude gate; silent frames are rejected before SRP-PHAT.
enable.smoothing	true or false	Enables circular-mean temporal smoothing over the azimuth history.
smoothing.window	5	Number of consecutive estimates over which the circular mean is computed.

Table 10: Configuration parameters for the speech localization node.

Topics Subscribed

All three nodes in the package subscribe to the same audio topic, `/naoqi_driver/audio`, as listed in Table 11.

Node	Topic Name	Message Type
speech.recognition	<code>/naoqi_driver/audio</code>	<code>naoqi_bridge_msgs/AudioBuffer</code>
sound.localization	<code>/naoqi_driver/audio</code>	<code>naoqi_bridge_msgs/AudioBuffer</code>
audio.recorder	<code>/naoqi_driver/audio</code>	<code>naoqi_bridge_msgs/AudioBuffer</code>

Table 11: Topics subscribed by the speech event package nodes.

Topics Published

The topics published by the speech event package nodes are listed in Table 12.

Topic Name	Message Type	Description
/speech_event/vad_speech_prob	std_msgs/Float32	Real-time Silero VAD speech probability for each 512-sample chunk.
/speech_event/text	std_msgs/String	Verbatim transcription published in standalone (continuous) mode.
/sound_localization/azimuth	std_msgs/Float32	Estimated horizontal azimuth (degrees) of the sound source.
/sound_localization/direction	geometry_msgs/Vector3Stamped	Unit direction vector of the estimated sound source.
/sound_localization/confidence	std_msgs/Float32	Confidence score for the current azimuth estimate.
/sound_localization/source_pose	geometry_msgs/PoseStamped	Estimated pose of the sound source in the robot frame.
/sound_localization/visualization	visualization_msgs/Marker	RViz arrow marker for visualizing the sound direction.

Table 12: Topics published by the speech event package nodes.

Action Server and Service Interfaces

Table 13 lists the action server and service interfaces exposed by the speech recognition node.

Interface	Name	Description
Action Server	/speech_recognition	dec_interfaces/action/SpeechRecognition action; accepts a goal with a configurable maximum wait duration and returns the verbatim transcription as the action result.
Service	/speech_event/set_enabled	std_srvs/SetBool service; pauses (false) or resumes (true) audio processing at runtime in either operating mode.

Table 13: Action server and service interfaces of the speech recognition node.

References

- [1] CSSR4Africa Deliverable D3.1: System Architecture. Technical report, Carnegie Mellon University Africa, 2025. Revision 2.6. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D3.1.pdf.
- [2] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022.
- [3] Ultralytics. Ultralytics yolo11. <https://github.com/ultralytics/ultralytics>, 2024. Accessed: March 2026.
- [4] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, 2022.
- [5] CSSR4Africa Deliverable D3.1: System Architecture. Technical report, Carnegie Mellon University Africa, 2025. Revision 2.6. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D3.1.pdf.
- [6] CSSR4Africa Deliverable D3.1: System Architecture. Technical report, Carnegie Mellon University Africa, 2025. Revision 2.6. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D3.1.pdf.
- [7] Roboflow. Supervision: Reusable computer vision tools. <https://github.com/roboflow/supervision>, 2024. Accessed: March 2026.
- [8] Thorsten Hempel, Ahmed A. Abdelrahman, and Ayoub Al-Hamadi. 6d rotation representation for unconstrained head pose estimation. In *IEEE International Conference on Image Processing (ICIP)*, 2022.
- [9] Chengcheng Wang, Wei He, Ying Nie, Jianyuan Guo, Chuanjian Liu, Yunhe Wang, and Kai Han. Gold-yolo: Efficient object detector via gather-and-distribute mechanism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] Silero Team. Silero vad: Pre-trained enterprise-grade voice activity detector. <https://github.com/snakers4/silero-vad>, 2021. Accessed: March 2026.
- [11] SYSTRAN. Faster-whisper: Fast inference engine for whisper using ctranslate2. <https://github.com/SYSTRAN/faster-whisper>, 2023. Accessed: March 2026.
- [12] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [13] Robin Scheibler, Eric Bezzam, and Ivan Dokmanić. Pyroomacoustics: A python package for audio room simulation and array processing algorithms. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018.

Principal Contributors

The main authors of this deliverable are as follows (in alphabetical order).

Yohannes Haile, Carnegie Mellon University Africa.

References

- [1] CSSR4Africa Deliverable D3.1: System Architecture. Technical report, Carnegie Mellon University Africa, 2025. Revision 2.6. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D3.1.pdf.
- [2] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022.
- [3] Ultralytics. Ultralytics yolo11. <https://github.com/ultralytics/ultralytics>, 2024. Accessed: March 2026.
- [4] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, 2022.
- [5] CSSR4Africa Deliverable D3.1: System Architecture. Technical report, Carnegie Mellon University Africa, 2025. Revision 2.6. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D3.1.pdf.
- [6] CSSR4Africa Deliverable D3.1: System Architecture. Technical report, Carnegie Mellon University Africa, 2025. Revision 2.6. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D3.1.pdf.
- [7] Roboflow. Supervision: Reusable computer vision tools. <https://github.com/roboflow/supervision>, 2024. Accessed: March 2026.
- [8] Thorsten Hempel, Ahmed A. Abdelrahman, and Ayoub Al-Hamadi. 6d rotation representation for unconstrained head pose estimation. In *IEEE International Conference on Image Processing (ICIP)*, 2022.
- [9] Chengcheng Wang, Wei He, Ying Nie, Jianyuan Guo, Chuanjian Liu, Yunhe Wang, and Kai Han. Gold-yolo: Efficient object detector via gather-and-distribute mechanism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] Silero Team. Silero vad: Pre-trained enterprise-grade voice activity detector. <https://github.com/snakers4/silero-vad>, 2021. Accessed: March 2026.
- [11] SYSTRAN. Faster-whisper: Fast inference engine for whisper using ctranslate2. <https://github.com/SYSTRAN/faster-whisper>, 2023. Accessed: March 2026.
- [12] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [13] Robin Scheibler, Eric Bezzam, and Ivan Dokmanić. Pyroomacoustics: A python package for audio room simulation and array processing algorithms. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018.

Document History

Version 1.0

First draft.
Yohannes Haile.
01 June 2026.

Version 1.1

Added the Robot Platform and Sensors overview (Figure 1); added the LiDAR and RealSense Sensor Mount subsection covering purpose, mounting interface, isometric render and physical assembly photos, material and manufacturing (ABS), and the power supply and companion electronics (XL4016 buck converter, power bank, and NVIDIA Jetson Nano); added bibliography references for the software and libraries used (YOLOv11, BYTETrack, ONNX Runtime, supervision, Gold-YOLO, SixDRepNet, Silero VAD, Faster-Whisper, Whisper, pyroomacoustics, ROS2, and SciPy).
Yohannes Haile.
17 July 2026.

Version 1.2

Updated the Person Detection and Face Detection design and implementation sections for their Python→C++ rewrite (`roclcpp_lifecycle::LifecycleNode` two-file packages, the native C++ `dec_common` ByteTrack port, and face detection's standalone-mode ByteTrack fallback); corrected the mutual-gaze test (yaw and pitch only) and added `prioritize_face_depth`.
Corrected both modules' configuration tables from `camelCase` to the real `snake_case` names, and their launch files, which were renamed and no longer start a camera or launch `person_detection` (`launch_camera` now defaults to `false`).
Noted that only `speech_recognition` is a ROS2 lifecycle node, and pinned every table in place with `[H]` (float), fixing the Face Detection *Topics Subscribed* table that had otherwise been stranded on its own page.
Yohannes Haile.
07 August 2026.

Version 1.3

Added Speech Event's `transcription_timeout_s` to the Speech Recognition configuration table, declared/read in code with a 5.0s default but previously missing from both the shipped YAML and this document.
Yohannes Haile.
13 August 2026.

Version 1.4

Added the ByteTrack algorithm diagram to the Person Detection module's Per-Frame Data Flow section, with an explicit figure reference at the tracking step it illustrates.
Added explicit `Figure~\ref{}/Table~\ref{}` callouts for the three file-structure figures and eight tables that previously lacked one (person detection, face detection, and speech event package sections), matching the referencing convention already used elsewhere in the document.
Yohannes Haile.
14 August 2026.